

Scaling and a Strictly Causal Continuous Cache for a Small GPT

LI Maoyuan

UID: 3035947946

DASE7506 Mini Project 1 · Formal seed 7506

September 2026

Abstract

This work studies how far the supplied small language model can be improved under the fixed MP1 protocol: training from random initialization on the supplied WikiText-2 training split, using the supplied BPE-2048 tokenizer, independent causal windows of 256 tokens, and bits per UTF-8 byte (BPB) as the metric. The final predictor combines two changes. First, the classroom GPT is scaled from width 128 and depth 4 to width 320 and depth 8, with 10 attention heads and 10,601,600 trainable parameters. Second, at evaluation time, its neural distribution is interpolated with a strictly causal continuous cache over earlier hidden states and observed successors in the current window. The cache resets for every independent example and never uses a future token or cross-window state.

The formal protocol separately measures four signed BPB changes: learning-rate horizon, architecture at equal processed targets, additional training, and cache interpolation with identical neural weights. At seed 7506, the initial 1,200-step baseline obtained 2.075891 validation BPB. Changing only its cosine schedule horizon from 1,200 to 4,800 steps improved BPB to 1.997825. The final frozen predictor obtained **1.598754 validation BPB**, a reduction of **0.477137 BPB (22.985%)** from the initial baseline, and **1.616003 test BPB**. Its paired score-loop time was 4.182842x the baseline, peak RSS was 1.517380 GiB, and uncompressed inference assets were 40.478294 MiB, all within the stated limits. Because formal comparisons use one seed, they are paired observations rather than evidence of statistical stability. Earlier exploratory work accessed the public test split; this is disclosed, and no claim is made that it was historically unseen.

1. Task, metric and baseline

Autoregressive language modelling assigns a conditional probability to every next token:

$$p(x_{1:T}) = \prod_{t=1}^{T-1} p(x_{t+1} \mid x_{\leq t}).$$

For tokenized text containing B raw UTF-8 bytes, the evaluator reports

$$\text{BPB} = -\frac{1}{B \ln 2} \sum_{t=1}^{T-1} \ln p(x_{t+1} \mid x_{\leq t}).$$

This is total next-token cross-entropy converted from nats to bits and divided by original byte count. Lower BPB means that the model assigns more probability to the observed continuation. Unlike token-level perplexity, BPB normalizes by the original text and is not directly changed by the number of tokens emitted by a tokenizer. The assignment nevertheless fixes the tokenizer, so every formal run has exactly the same token and byte denominators.

The supplied baseline is a decoder-only Transformer [1]: width 128, four pre-normalized blocks, four attention heads, context 256, and 1,088,256 parameters. Each block contains causal multi-head self-attention, a four-times- width GELU MLP and two residual additions. Learned token and position embeddings form the input, a final LayerNorm precedes the output, and token-embedding and output-projection weights are tied. Training minimizes next-token cross-entropy from random initialization.

This project asks two questions. First, when seed, batches, optimizer, learning-rate path and processed targets are matched, does an approximately ten-million-parameter wider, moderately deep Transformer improve validation BPB? Second, with the neural checkpoint fixed, can a small window-local cache exploit repetition without violating causality?

2. Method

2.1 Scaled GPT

The proposed neural model preserves the supplied block formulation and changes only capacity:

Component	Baseline	Proposed
Vocabulary	2,048	2,048
Context	256	256
Width	128	320
Blocks	4	8
Attention heads	4	10
Head dimension	32	32
MLP expansion	4x	4x
Parameters	1,088,256	10,601,600

For ids $x_1, \dots, x_L$, learned token and position embeddings pass through eight causal blocks. The normalized final state h_t gives the neural distribution

$$p_{\text{NN}}(y \mid x_{\leq t}) = \text{softmax}(Wh_t)_y.$$

Ten heads preserve the baseline head dimension: $10 \times 32 = 320$. This avoids confounding greater model width with a different per-head dimension. The cache does not participate in training and adds no learned parameters.

The parameter counts can be audited directly from the architecture. With vocabulary V , context length C , width d and N blocks, tied input/output embeddings give

$$P(V, C, d, N) = Vd + Cd + N(12d^2 + 13d) + 2d.$$

Within each block, Q/K/V and the attention output contribute $4d^2$, while the two MLP projections contribute $8d^2$; biases and two LayerNorms account for the $13d$ term. Substitution yields 1,088,256 parameters for $(d, N) = (128, 4)$ and 10,601,600 for $(320, 8)$. Because width, depth and head count change together, the formal same-target control identifies the effect of this capacity/shape bundle, not a separate causal effect for any one dimension.

2.2 Strictly causal continuous cache

The method adapts the continuous-cache idea of Grave, Joulin and Usunier [5]. For query position t , allowed memories are

$$\mathcal{M}_t = \{m : \max(0, t - K) \leq m < t\}, \quad K = 255.$$

Memory position m stores hidden state h_m with observed successor x_{m+1} . Since $m < t$, the latest stored successor is x_t , already present in the allowed query prefix. The target x_{t+1} is never accessed. After L_2 -normalization, memory weights are

$$a_{t,m} = \frac{\exp(\theta \cos(h_t, h_m))}{\sum_{j \in \mathcal{M}_t} \exp(\theta \cos(h_t, h_j))}, \quad \theta = 13,$$

and the cache distribution is

$$p_{\text{cache}}(y \mid x_{\leq t}) = \sum_{m \in \mathcal{M}_t} a_{t,m} \mathbf{1}[x_{m+1} = y].$$

The frozen output distribution is

$$p_{\text{final}} = (1 - \lambda)p_{\text{NN}} + \lambda p_{\text{cache}}, \quad \lambda = 0.065.$$

The first position has no memory and uses the neural distribution alone. A strict lower-triangular mask is applied before softmax. Temporary tensors are local to one `predict_log_probs` call, so there is no state across examples, windows or evaluations. The cache can help when similar recent contexts have similar successors; it can hurt when hidden similarity retrieves an irrelevant successor. The small interpolation weight limits that risk, and an exact cache-off/on comparison measures its net effect.

Causality follows from the index constraint rather than from an informal assumption about implementation. For a query at t , every memory satisfies $m < t$, so its largest possible successor index is $m + 1 = t$; that token is already inside $x_{\leq t}$. The lower-triangular mask is applied to

similarity logits before softmax, hence a forbidden location receives exactly zero probability. With $K = 255$, all useful earlier positions of a 256-token window are available, but none from another window are retained.

For the observed next token y^* , interpolation helps exactly when $p_{\text{cache}}(y^*) > p_{\text{NN}}(y^*)$ and hurts when the inequality is reversed. If the cache gives y^* zero mass, the neural probability is only scaled by $1 - \lambda = 0.935$, corresponding to a worst-case extra penalty of $-\log_2(0.935) = 0.09696$ bits per token. This bounded downside is one reason for using a small validation-selected interpolation weight.

2.3 Optimizer and cosine trajectory

All formal runs use AdamW [4], whose decoupled weight decay is distinct from an L_2 penalty under adaptive optimization. The peak learning rate is 10^{-3} , warmup is 100 updates, minimum/peak ratio is 0.1, weight decay is 0.1, and gradient norm is clipped at 1.0.

For zero-indexed update s and horizon H , the implemented learning rate is

$$\eta_s = 10^{-3} \min\left(1, \frac{s+1}{100}\right) \left[0.1 + \frac{0.9}{2} \left(1 + \cos \frac{\pi s}{H}\right)\right].$$

The warmup factor and cosine factor are multiplied, and the final 0.1 term prevents decay below one tenth of the peak. Crucially, H is a configuration of the trajectory rather than the number of updates actually executed.

The post-warmup learning rate follows a single cosine decay inspired by SGDR [3], but uses no restart. A schedule horizon is defined independently of the actual stop. Thus step 1,200 on a 4,800-step trajectory has a higher learning rate than the end of a 1,200-step trajectory. The matched baseline controls this otherwise hidden training difference. Concretely, the recorded step-1,200 learning rates are 1.0000×10^{-4} for B0 and 8.6841×10^{-4} for B1/P0.

3. Architecture and schedule selection

3.1 Shape at an approximately fixed parameter budget

Before the formal protocol, three approximately ten-million-parameter shapes were trained for 600 updates using exploratory seed 17. Model size, update count, batch size and training data were closely matched:

Candidate	Parameters	Targets	Validation BPB
width 256, 12 blocks, 8 heads	10,067,456	4,915,200	2.137945
width 288, 10 blocks, 9 heads	10,654,848	4,915,200	2.107498
width 320, 8 blocks, 10 heads	10,601,600	4,915,200	2.076717

The width-320, eight-block candidate was best in this bounded comparison. One plausible explanation is that the shallow-wide allocation exposes more channel capacity to every attention and MLP operation while avoiding the optimization depth of 10 or 12 blocks at a short budget. This is an empirical explanation, not a universal rule: the search covered only three shapes, one

seed and one short training budget.

3.2 Why stop at 3,000 on a 4,800-step horizon?

Exploratory width-320 runs compared the supplied cosine trajectory with a warmup-stable-decay (WSD) alternative over a nominal 4,800-step horizon. At the intended early checkpoint, cosine was better: step-3,000 validation BPB was 1.644901, versus 1.662993 for WSD. Cosine then changed to 1.645779 at step 3,600, 1.659134 at 4,200 and 1.676923 at 4,800. WSD improved later and reached 1.650036 at step 4,200, but did not beat cosine's step-3,000 result. This pattern suggests that prolonged training after the cosine optimum overfit or moved away from a good validation basin, while WSD's late decay recovered more slowly.

The project did not exhaustively tune WSD's stable/decay split. Given that MP1 counts for 10% of the course and the best observed checkpoint was already cosine step 3,000, the formal design prioritizes an auditable control matrix over a larger schedule search. It performs exactly 3,000 updates but keeps the 4,800-step horizon so that the selected learning-rate path is reproduced and the step-1,200 proposed checkpoint can be compared with a baseline on the same path. The stop and horizon were fixed before the formal run.

The cache constants were likewise frozen from earlier validation grids. The best explored width-320 setting at step 3,000 was window 255, temperature 13 and interpolation 0.065. Window 255 is the maximum useful history for a 256-token independent window: a query can have at most 255 earlier positions. Temperature controls concentration over similar states, while interpolation sets how much probability mass the cache may redirect. These values are validation-selected hyperparameters, not learned parameters.

4. Formal protocol and controls

Training uses only the supplied train text. Development uses the supplied validation text. The formal development directory physically omits the test file; its trainer and validation driver have no test-split option. Tokenizer, text, official evaluator and causal window construction are hash-checked. Each update processes $32 \times 256 = 8,192$ targets. All formal runs use seed 7506, CPU FP32 and four threads.

ID	Model	Updates / LR horizon	Targets	Cache
B0	baseline 128x4	1,200 / 1,200	9,830,400	off
B1	baseline 128x4	1,200 / 4,800	9,830,400	off
P0-1200	proposed 320x8 snapshot	1,200 / 4,800	9,830,400	off
P0-3000-off	proposed final weights	3,000 / 4,800	24,576,000	off
P0-3000-on	identical final weights	3,000 / 4,800	24,576,000	255/13/.065

B1 and P0-1200 match seed, batch-start sequence, optimizer, schedule and target count. Cache-on and cache-off artifacts contain exactly the same state dict. Negative signed differences are improvements:

$$\Delta_{\text{schedule}} = \text{BPB}(B1) - \text{BPB}(B0),$$

$$\Delta_{\text{architecture}} = \text{BPB}(P0_{1200}) - \text{BPB}(B1),$$

$$\Delta_{\text{training}} = \text{BPB}(P0_{3000,off}) - \text{BPB}(P0_{1200}),$$

$$\Delta_{\text{cache}} = \text{BPB}(P0_{3000,on}) - \text{BPB}(P0_{3000,off}).$$

Their sum is the final validation change from B0. Only the architecture delta is a same-target architecture comparison; only the cache delta isolates the cache; the training delta deliberately includes 14,745,600 additional targets.

The training-state checkpoint records model, AdamW state, global step, target count, batch generator, Torch RNG and history. Synthetic testing confirms a bit-exact resume despite stochastic dropout support. Additional tests check causality, normalized output probabilities, example independence, state reset, gradient flow, data isolation and frozen-checkpoint identity.

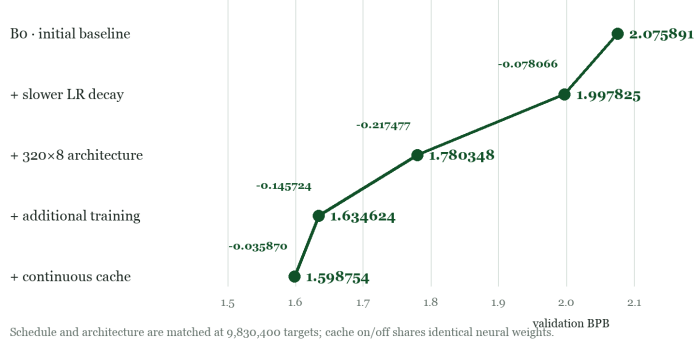
5. Results

Predictor	Seed	Updates / horizon	Targets	Cache	Validation BPB	Test BPB
Initial baseline B0	7506	1,200 / 1,200	9,830,400	off	2.075891	not run
Matched baseline B1	7506	1,200 / 4,800	9,830,400	off	1.997825	not run
Proposed same-target	7506	1,200 / 4,800	9,830,400	off	1.780348	not run
Proposed longer	7506	3,000 / 4,800	24,576,000	off	1.634624	not run
Frozen submission	7506	3,000 / 4,800	24,576,000	255/13/.065	1.598754	1.616003

Controlled effect	Signed validation-BPB delta
Schedule horizon	-0.078066
Architecture at equal targets	-0.217477
Additional training	-0.145724
Cache with identical weights	-0.035870
Total versus B0	-0.477137

Controlled validation-BPB decomposition

Lower is better - formal seed 7506



The schedule result is already concrete: $1.997825 - 2.075891 = -0.078066$ BPB. It shows why B0 alone is not a fair architecture control: the slow 4,800-step trajectory yields a material gain without changing model or targets.

At equal targets, P0-1200 improves on B1 by 0.217477 BPB for seed 7506. This is the matched evidence for architecture/capacity, not a multi-seed stability claim. Continuing the same P0 trajectory from 1,200 to the prespecified 3,000 updates contributes a further 0.145724 BPB, and enabling the cache on the exact same step-3,000 neural weights contributes 0.035870 BPB. The four signed changes sum to -0.477137 BPB, or a 22.9847% reduction relative to B0. The cache-off/on comparison is the cleanest mechanism ablation because the two artifacts share the same source checkpoint and differ only in frozen cache configuration.

The stored per-window validation losses permit a paired descriptive check. Cache-on lowers summed NLL in 1,259 of 1,472 windows (85.53%) and raises it in 213 (14.47%). The paired change has median -13.978 nats per window and sums to -28,542.748 nats over validation. Thus the aggregate cache gain is distributed across most windows rather than arising from only a handful of outliers, while the harmed windows confirm that retrieval is not uniformly beneficial. These windows are contiguous pieces of one corpus, so the counts are descriptive and are not treated as independent samples or a significance test.

6. Computation and resource trade-offs

The proposed network has 9.74 times as many trainable parameters as B0, but matrix operations use larger, more efficient kernels; measured latency, rather than parameter ratio, determines compliance. The cache computes a dense $L \times L$ similarity matrix and a temporary $L \times V$ distribution for $L \leq 256$ and $V = 2,048$. It adds no persistent retrieval database and no trainable parameter. The quality gain is therefore exchanged for neural matrix-multiplication cost and a bounded amount of temporary RAM.

Paired resource measurements use the same computer, validation workload, FP32 and four threads. The official test BPB is measured separately only for the frozen final predictor.

Predictor	Score seconds	Relative time	Peak RSS	Inference assets
B0 paired local baseline	8.209438 s	1.00x	1.509064 GiB	4.170731 MiB
Frozen 320x8 + cache	34.338785 s	4.182842x	1.517380 GiB	40.478294 MiB
MP1 limit	--	5.00x	4.00 GiB	64.00 MiB

The common evaluator's score-loop time gives a 4.182842x ratio, below the 5x limit. The final predictor also remains below the 4-GiB RSS and 64-MiB asset limits. These are paired measurements on this machine, corpus, precision and thread count; they should not be generalized into hardware-independent speed claims. As a secondary end-to-end observation, the external elapsed times were 16.55 s for B0 and 42.88 s for cache-on P0 (2.590937x). The one formal test obtained 1.6160026140038766 BPB in 41.520049 scorer seconds and 50.96 external elapsed seconds, with 1.550232 GiB peak RSS. It was not used for selection.

7. Cost, limitations and integrity

7.1 Training and search cost

The two formal baseline jobs consumed 736.705 training seconds and 765.958 whole-process seconds. Formal P0 consumed 4,775.762 training seconds and 4,843.256 process seconds. Three separate pre-test resource-scoring commands (B0, P0 cache-off and P0 cache-on) consumed 100.55 external elapsed seconds; the freeze operation and contract-test commands added approximately 12.874 seconds. The single final-test job adds 50.96 external elapsed seconds. These scopes are stated separately because score-loop seconds, whole-process seconds and external elapsed seconds are not interchangeable. They are job-time sums, not necessarily elapsed wall-clock time.

Prior exploration is also included rather than hidden. Fourteen surviving training runs sum to 38,326.481 training seconds, 39,908.333 process seconds and 157,532,160 targets. Ten separately timed evaluations add 157.053 seconds. The documented exploratory lower bound is therefore 40,065.386 job-seconds, or 11.1293 job-hours. Six cache grids contain 373 tried configurations, but their elapsed time was not stored; it is marked unknown rather than zero. Failed or interrupted jobs without artifacts cannot be reconstructed completely.

Cost category	Runs / trials	Recorded job-time
Exploratory training with metrics	14 runs	39,908.333 process s
Standalone exploratory evaluation	10 runs	157.053 s
Cache grids	373 settings	time not recorded
Formal B0 and B1	2 runs	765.958 process s
Formal P0	1 run	4,843.256 process s
Pre-test resource scoring	3 jobs	100.55 external elapsed s
Freeze and contract tests	multiple commands	approximately 12.874 external elapsed s
Single formal final test	1 job	50.96 external elapsed s

Including the final test, the directly recorded formal total is approximately 5,773.598 job-seconds when the differently scoped entries above are summed. Including the documented exploratory lower

bound gives approximately 45,838.984 job-seconds. These totals remain lower bounds because cache-grid time and failed jobs without artifacts are unknown.

7.2 Single seed and bounded search

Every formal comparison uses seed 7506. Matching seed and batch order removes one source of noise from paired differences, but one run cannot estimate variance over initialization. No error bars, confidence intervals, statistical- significance claims, or assertions of robust improvement are made. The result describes the submitted predictor under this protocol.

Architecture search covered three shapes at one short budget. Cache parameters were tuned on earlier validation data and may be validation-specific. WSD's stable/decay split was not exhaustively searched. These choices deliberately trade breadth for a small, reproducible formal matrix appropriate to a 10%-of- course mini project.

7.3 Historical test exposure

Before preregistration, five test evaluations were run on discarded exploratory predictors, with BPB values 2.010090, 2.101265, 2.091555, 1.622126 and 1.655538. This was a protocol mistake. Extracting a fresh ZIP does not erase information already observed, so this report does not claim that test was historically unseen.

The mitigation is prospective. A clean formal-development tree omitted the test file physically. B0, B1, P0, seed 7506, the 3,000-step stop, the 4,800-step horizon, and cache 255/13/.065 were committed before formal results. The final checkpoint and code hashes were frozen before the one new formal full-test evaluation, and no formal setting was changed in response to that score. This reduces further test-driven adaptation but cannot restore a statistically pristine holdout.

7.4 AI assistance

OpenAI Codex/ChatGPT was used substantively as a coding and discussion assistant. It helped explain the baseline and BPB, search and summarize published methods, propose and critique controls, scaffold and review code and tests, orchestrate commands, diagnose issues, audit costs and draft documentation. I reviewed the implementation and explanations, ran and interpreted the experiments, checked mechanisms against code, and remain responsible for design, citations, claims, disclosure and submission. AI output was not treated as experimental evidence; each reported number is tied to an artifact and command.

7.5 Reproducibility map

The repository README gives exact setup, training, validation, resource measurement, freeze and final-scoring commands. A machine-readable record binds the selected checkpoint to hashes of the code, tokenizer, training/validation text and official evaluator. The submission manifest lists the distributable files and checksums. The checkpoint bundle restores without retraining, and the 24 contract tests cover evaluation equivalence, causality, state reset, data isolation and bit-exact training resume. This separates three claims that are otherwise easy to conflate: the experiment can be rerun from random initialization, the submitted predictor can be scored directly, and the reported artifact can be matched to the frozen evidence.

8. Conclusion

The final predictor scales the baseline GPT to width 320 and eight blocks, trains it along a prespecified longer cosine trajectory, and applies a bounded continuous cache inside each independent window. The formal design prevents the headline BPB gain from being assigned to one idea: schedule, same-target architecture, additional targets and cache each have a distinct controlled difference. The submitted predictor changes validation BPB by -0.477137 (22.9847%) from B0 and obtains 1.616003 official test BPB at 4.182842 times paired baseline CPU score-loop time.

The strongest causal evidence is the cache ablation, because it changes no neural weight. The same-target comparison supports an architecture effect for seed 7506, while the longer-run comparison quantifies the contribution of extra training. Conclusions remain bounded by the single seed, narrow architecture search, hardware-specific timing and disclosed historical test exposure.

References

- [1] A. Vaswani et al. “Attention Is All You Need.” *NeurIPS*, 2017. Online paper.
- [2] S. Merity, C. Xiong, J. Bradbury, and R. Socher. “Pointer Sentinel Mixture Models.” *ICLR*, 2017. Online paper.
- [3] I. Loshchilov and F. Hutter. “SGDR: Stochastic Gradient Descent with Warm Restarts.” *ICLR*, 2017. Online paper.
- [4] I. Loshchilov and F. Hutter. “Decoupled Weight Decay Regularization.” *ICLR*, 2019. Online paper.
- [5] E. Grave, A. Joulin, and N. Usunier. “Improving Neural Language Models with a Continuous Cache.” *ICLR*, 2017. Online paper.